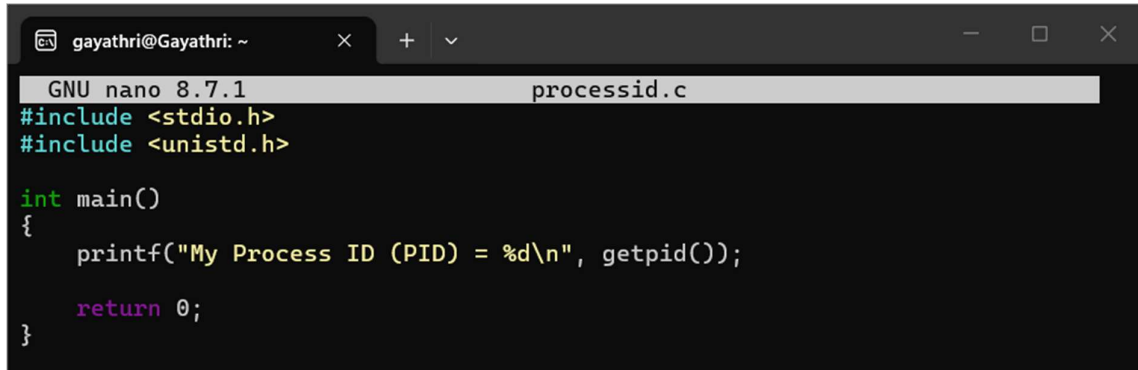


OS PROCESS CONCEPTS CODES AND OUTPUTS

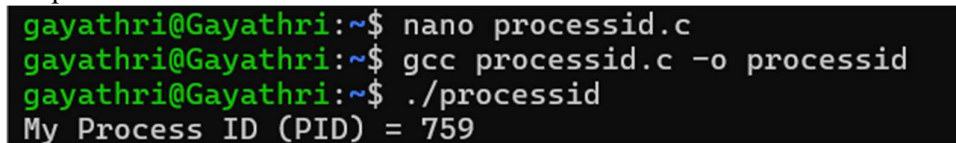
1.Processed:

Code:



```
gayathri@Gayathri: ~  
GNU nano 8.7.1 processid.c  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    printf("My Process ID (PID) = %d\n", getpid());  
    return 0;  
}
```

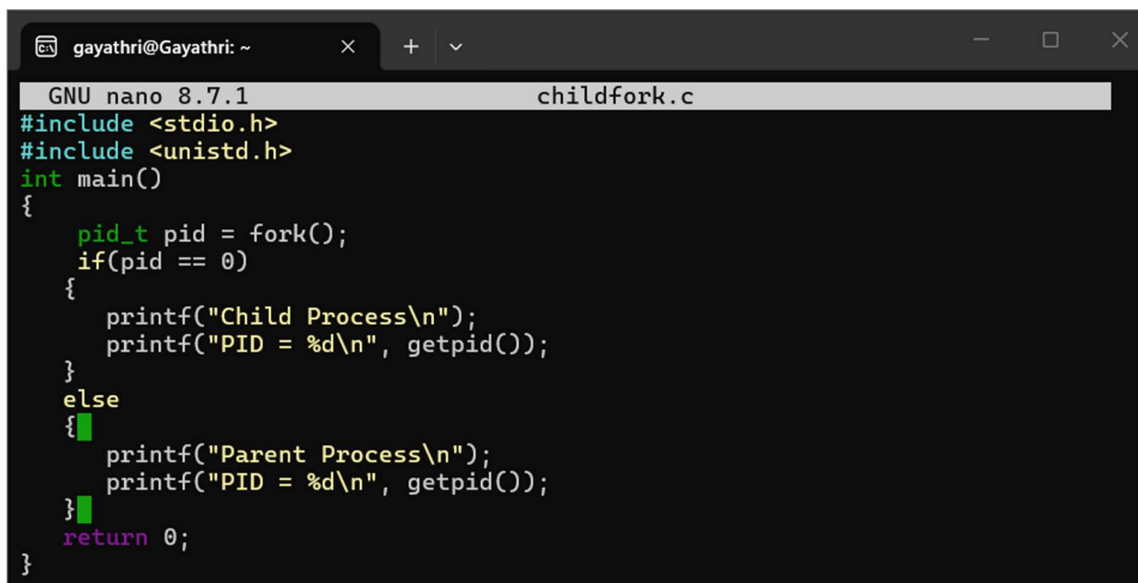
Output:



```
gayathri@Gayathri:~$ nano processid.c  
gayathri@Gayathri:~$ gcc processid.c -o processid  
gayathri@Gayathri:~$ ./processid  
My Process ID (PID) = 759
```

2. creating child using fork()

Code:



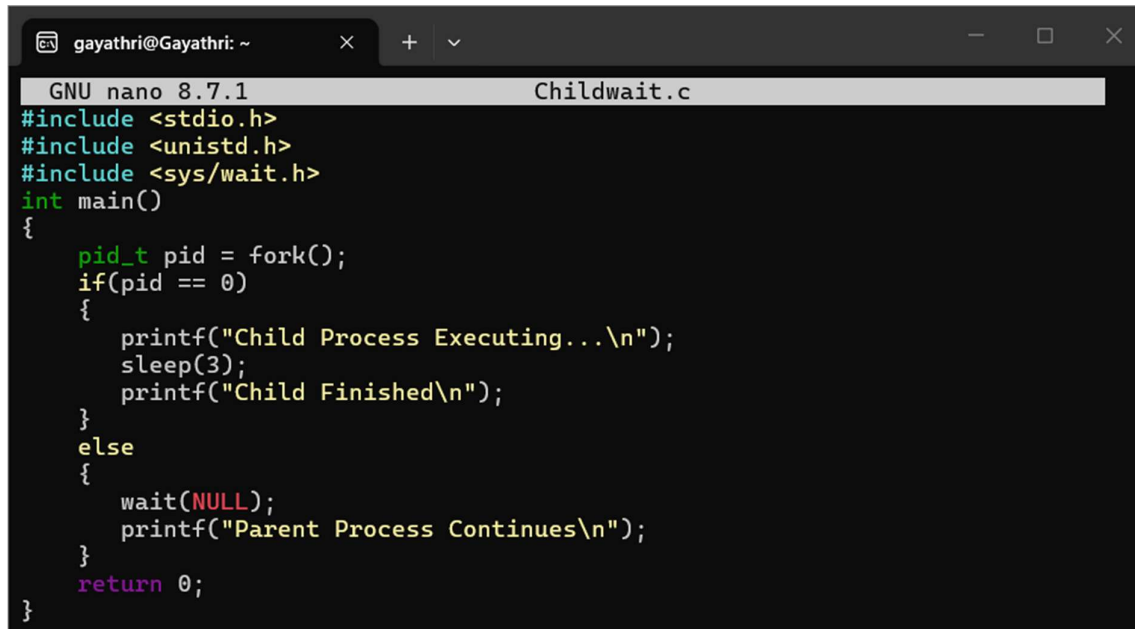
```
gayathri@Gayathri: ~  
GNU nano 8.7.1 childfork.c  
#include <stdio.h>  
#include <unistd.h>  
int main()  
{  
    pid_t pid = fork();  
    if(pid == 0)  
    {  
        printf("Child Process\n");  
        printf("PID = %d\n", getpid());  
    }  
    else  
    {  
        printf("Parent Process\n");  
        printf("PID = %d\n", getpid());  
    }  
    return 0;  
}
```

Output:

```
gayathri@Gayathri:~$ nano childfork.c
gayathri@Gayathri:~$ gcc childfork.c -o childfork
gayathri@Gayathri:~$ ./childfork
Parent Process
PID = 818
Child Process
PID = 819
```

3. Parent Waits for Child (wait())

Code:



```
GNU nano 8.7.1 Childwait.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child Process Executing...\n");
        sleep(3);
        printf("Child Finished\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Process Continues\n");
    }
    return 0;
}
```

Output:

```
gayathri@Gayathri:~$ nano Childwait.c
gayathri@Gayathri:~$ gcc Childwait.c -o Childwait
gayathri@Gayathri:~$ ./Childwait
Child Process Executing...
Child Finished
Parent Process Continues
```

4. Multiple Child Processes

Code:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 MultipleChild.c  
#include <stdio.h>  
#include <unistd.h>  
int main()  
{  
    for(int i = 1; i <=3; i++)  
    {  
        if(fork()==0)  
        {  
            printf("Child %d PID = %d\n", i, getpid());  
            return 0;  
        }  
    }  
    return 0;  
}
```

Output:

```
gayathri@Gayathri:~$ nano MultipleChild.c  
gayathri@Gayathri:~$ gcc MultipleChil.c -o MultipleChild  
cc1: fatal error: MultipleChil.c: No such file or directory  
compilation terminated.  
gayathri@Gayathri:~$ gcc MultipleChild.c -o MultipleChild  
gayathri@Gayathri:~$ ./MultipleChild  
Child 1 PID = 1294  
Child 2 PID = 1295  
Child 3 PID = 1296
```

5. Execute linux command using execlp()

Code:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 execlp.c  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    printf("Before exec()\n");  
  
    execlp("ls", "ls", "-l", NULL);  
  
    printf("This will not execute.\n");  
  
    return 0;  
}
```

Output:

```
gayathri@Gayathri: ~  
gayathri@Gayathri:~$ nano exclp.c  
gayathri@Gayathri:~$ gcc exclp.c -o exclp  
gayathri@Gayathri:~$ ./exclp  
Before exec()  
total 284  
-rwxr-xr-x 1 gayathri gayathri 16080 Aug 12 07:00 Childwait  
-rw-r--r-- 1 gayathri gayathri 327 Aug 12 07:02 Childwait.c  
-rwxr-xr-x 1 gayathri gayathri 16176 Aug 6 04:02 Copy1to0ther  
-rw-r--r-- 1 gayathri gayathri 534 Aug 6 04:02 Copy1to0ther.c  
-rwxr-xr-x 1 gayathri gayathri 15952 Aug 7 15:23 Hello  
-rw-r--r-- 1 gayathri gayathri 94 Aug 7 15:22 Hello.c  
-rwxr-xr-x 1 gayathri gayathri 16304 Aug 6 03:58 Linux  
-rw-r--r-- 1 gayathri gayathri 562 Aug 6 04:00 Linux.c  
-rwxr-xr-x 1 gayathri gayathri 16048 Aug 12 08:07 MultipleChild  
-rw-r--r-- 1 gayathri gayathri 223 Aug 12 08:08 MultipleChild.c  
drwxr-xr-x 2 gayathri gayathri 4096 Jul 30 04:18 OSLAB  
-rwxr-xr-x 1 gayathri gayathri 16000 Aug 1 05:44 ParentChild  
-rw-r--r-- 1 gayathri gayathri 174 Aug 1 05:49 ParentChild.c  
-rwxr-xr-x 1 gayathri gayathri 16040 Aug 1 06:05 Pcpid  
-rw-r--r-- 1 gayathri gayathri 221 Aug 1 06:06 Pcpid.c  
-rwxr-xr-x 1 gayathri gayathri 16032 Aug 1 06:07 Pcrun  
-rw-r--r-- 1 gayathri gayathri 211 Aug 1 06:07 Pcrun.c  
-rwxr-xr-x 1 gayathri gayathri 16080 Aug 12 06:50 childfork  
-rw-r--r-- 1 gayathri gayathri 281 Aug 12 06:51 childfork.c  
-rwxr-xr-x 1 gayathri gayathri 16176 Aug 6 04:03 copyfile  
-rw-r--r-- 1 gayathri gayathri 534 Aug 6 04:06 copyfile.c  
-rwxr-xr-x 1 gayathri gayathri 15992 Aug 13 04:09 exclp  
-rw-r--r-- 1 gayathri gayathri 179 Aug 13 04:08 exclp.c  
-rw-r--r-- 1 gayathri gayathri 0 Jul 29 06:32 file1.tx  
-rw-r--r-- 1 gayathri gayathri 6 Jul 29 06:35 file1.txt  
-rwxr-xr-x 1 gayathri gayathri 15992 Aug 7 15:37 fork  
-rw-r--r-- 1 gayathri gayathri 90 Aug 7 15:36 fork.c  
-rw-r--r-- 1 gayathri gayathri 90 Aug 7 15:37 "fork.c"  
-rw-r--r-- 1 gayathri gayathri 0 Jul 29 07:03 hello.c  
-rw-r--r-- 1 gayathri gayathri 425 Aug 5 08:29 linux.c  
-rw----- 1 gayathri gayathri 6 Jul 29 06:34 nano.2179.save  
-rw----- 1 gayathri gayathri 6 Jul 29 06:34 nano.2238.save  
-rwxr-xr-x 1 gayathri gayathri 16000 Aug 12 06:41 processid  
-rw-r--r-- 1 gayathri gayathri 122 Aug 12 06:42 processid.c
```

6. Parent and Child Executing Concurrently

Code:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 pcec.c  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    fork();  
  
    for(int i=1; i<=5; i++)  
    {  
        printf("PID: %d Count: %d\n", getpid(), i);  
        sleep(1);  
    }  
  
    return 0;  
}
```

Output:

```
gayathri@Gayathri:~$ nano pcec.c  
gayathri@Gayathri:~$ gcc pcec.c -o pcec  
gayathri@Gayathri:~$ ./pcec  
PID: 687 Count: 1  
PID: 688 Count: 1  
PID: 687 Count: 2  
PID: 688 Count: 2  
PID: 688 Count: 3  
PID: 687 Count: 3  
PID: 687 Count: 4  
PID: 688 Count: 4  
PID: 687 Count: 5  
PID: 688 Count: 5
```

7. Zombie Process

Code:

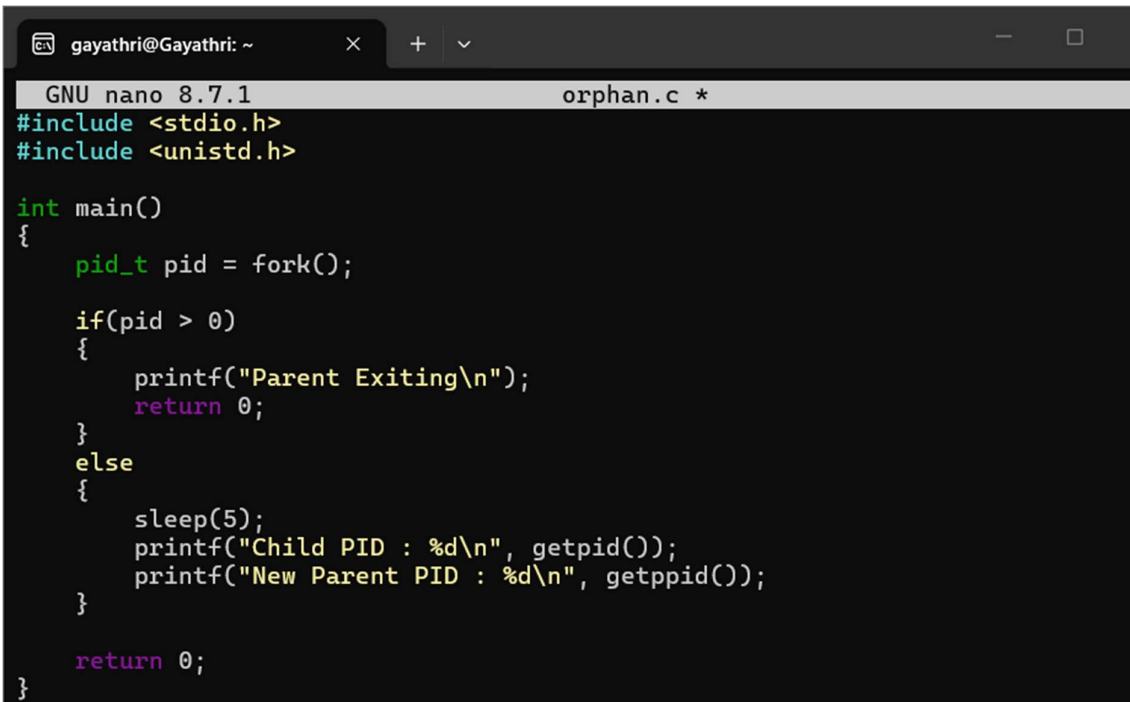
```
gayathri@Gayathri: ~  
GNU nano 8.7.1 zombie.c *  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    pid_t pid = fork();  
  
    if(pid == 0)  
    {  
        printf("Child Exiting\n");  
        return 0;  
    }  
    else  
    {  
        sleep(15);  
        printf("Parent Finished\n");  
    }  
  
    return 0;  
}
```

Output:

```
gayathri@Gayathri:~$ nano pcec.c
gayathri@Gayathri:~$ nano zombie.c
gayathri@Gayathri:~$ gcc zombie.c -o zombie
gayathri@Gayathri:~$ ./zombie
Child Exiting
Parent Finished
```

8. OrphanProcess

Code:



```
GNU nano 8.7.1 orphan.c *
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid > 0)
    {
        printf("Parent Exiting\n");
        return 0;
    }
    else
    {
        sleep(5);
        printf("Child PID : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }

    return 0;
}
```

Output:

```
gayathri@Gayathri:~$ nano orphan.c
gayathri@Gayathri:~$ gcc orphan.c -o orphan
gayathri@Gayathri:~$ ./orphan
Parent Exiting
gayathri@Gayathri:~$ Child PID : 938
New Parent PID : 342
```